Operating Systems
Week 8
Lecture/Discussion 8

Junaid Abdul Wahid

Previous

- Scheduling Algorithms (SJF Preemptive- Round Robin)

Today Discussion

- Continue slides from Lecture (Deadlock Prevention)

Deadlock Characterization

Deadlock can arise if four conditions hold simultaneously.

Mutual exclusion: only one thread at a time can use a resource

Hold and wait: a thread holding at least one resource is waiting to acquire additional resources held by other threads

No preemption: a resource can be released only voluntarily by the thread holding it, after that thread has completed its task

Circular wait: there exists a set $\{T_0, T_1, \ldots, T_n\}$ of waiting threads such that T_0 is waiting for a resource that is held by T_1, T_1 is waiting for a resource that is held by T_2, …, T_{n-1} is waiting for a resource that is held by T_n, and T_n is waiting for a resource that is held by T_0.

Deadlock Characterization

Imagine four friends (**processes**) each needing two chopsticks (**resources**) to eat. If each grabs one chopstick and waits for the other, no one eats – this is a deadlock.

Resource-Allocation Graph

A set of vertices V and a set of edges E.

V is partitioned into two types:

$T = \{T_1, T_2, \ldots, T_n\}$, the set consisting of all the threads in the system.

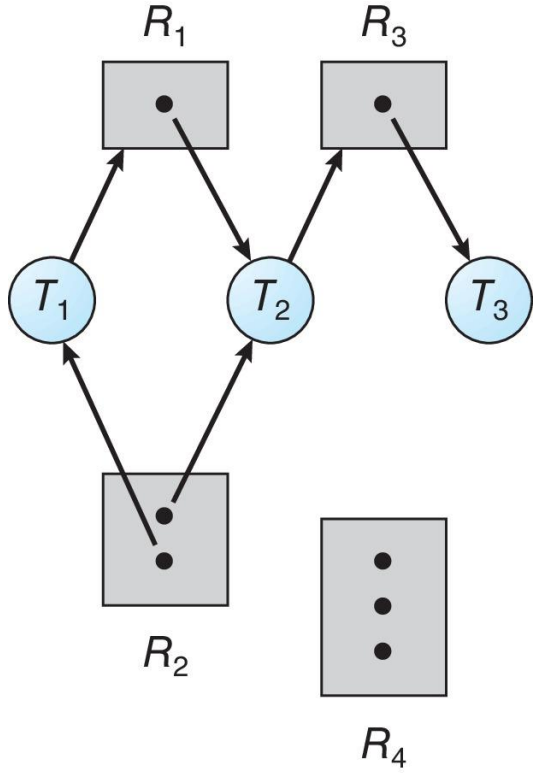
$R = \{R_1, R_2, \ldots, R_m\}$, the set consisting of all resource types in the system

request edge – directed edge $T_i \rightarrow R_j$

assignment edge – directed edge $R_j \rightarrow T_i$

Resource Allocation Graph Example

One instance of R_1
Two instances of R_2
One instance of R_3
Three instance of R_4
T_1 holds one instance of R_2 and is waiting for an instance of R_1
T_2 holds one instance of R_1, one instance of R_2, and is waiting for an instance of R_3
T_3 is holds one instance of R_3



Methods for Handling Deadlocks

Ensure that the system will **never** enter a deadlock state:

- Deadlock prevention
- Deadlock avoidance

Allow the system to enter a deadlock state and then recover

Ignore the problem and pretend that deadlocks never occur in the system.

Deadlock Prevention

1. Prevent Hold and Wait
Require a process to request all required resources at once.

2. Prevent No Preemption
If a process is holding some resources and requests another that's not available, force it to release the held ones.

3. Prevent Circular Wait
Impose a global ordering of all resources. Processes can only request resources in increasing order.
🧠 Easy to implement with IDs or priorities.

Deadlock Prevention

Invalidate one of the four necessary conditions for deadlock:

Mutual Exclusion – not required for sharable resources (e.g., read-only files); must hold for non-sharable resources

Hold and Wait – must guarantee that whenever a thread requests a resource, it does not hold any other resources

- Require threads to request and be allocated all its resources before it begins execution or allow thread to request resources only when the thread has none allocated to it.
- Low resource utilization; starvation possible

Deadlock Prevention (Cont.)

No Preemption:

If a process that is holding some resources requests another resource that cannot be immediately allocated to it, then all resources currently being held are released

Preempted resources are added to the list of resources for which the thread is waiting

Thread will be restarted only when it can regain its old resources, as well as the new ones that it is requesting

Circular Wait:

Impose a total ordering of all resource types, and require that each thread requests resources in an increasing order of enumeration

Deadlock Avoidance

Requires that the system has some additional ***a priori*** information available

Simplest and most useful model requires that each thread declare the ***maximum number*** of resources of each type that it may need

The deadlock-avoidance algorithm dynamically examines the resource-allocation state to ensure that there can never be a circular-wait condition

Resource-allocation *state* is defined by the number of available and allocated resources, and the maximum demands of the processes

Safe State

When a thread requests an available resource, system must decide if immediate allocation leaves the system in a safe state
System is in **safe state** if there exists a sequence $<T_1, T_2, \ldots, T_n>$ of ALL the threads in the systems such that for each T_i, the resources that T_i can still request can be satisfied by currently available resources + resources held by all the T_j, with $j < I$
That is:

- If T_i resource needs are not immediately available, then T_i can wait until all T_j have finished
- When T_j is finished, T_i can obtain needed resources, execute, return allocated resources, and terminate
- When T_i terminates, T_{i+1} can obtain its needed resources, and so on

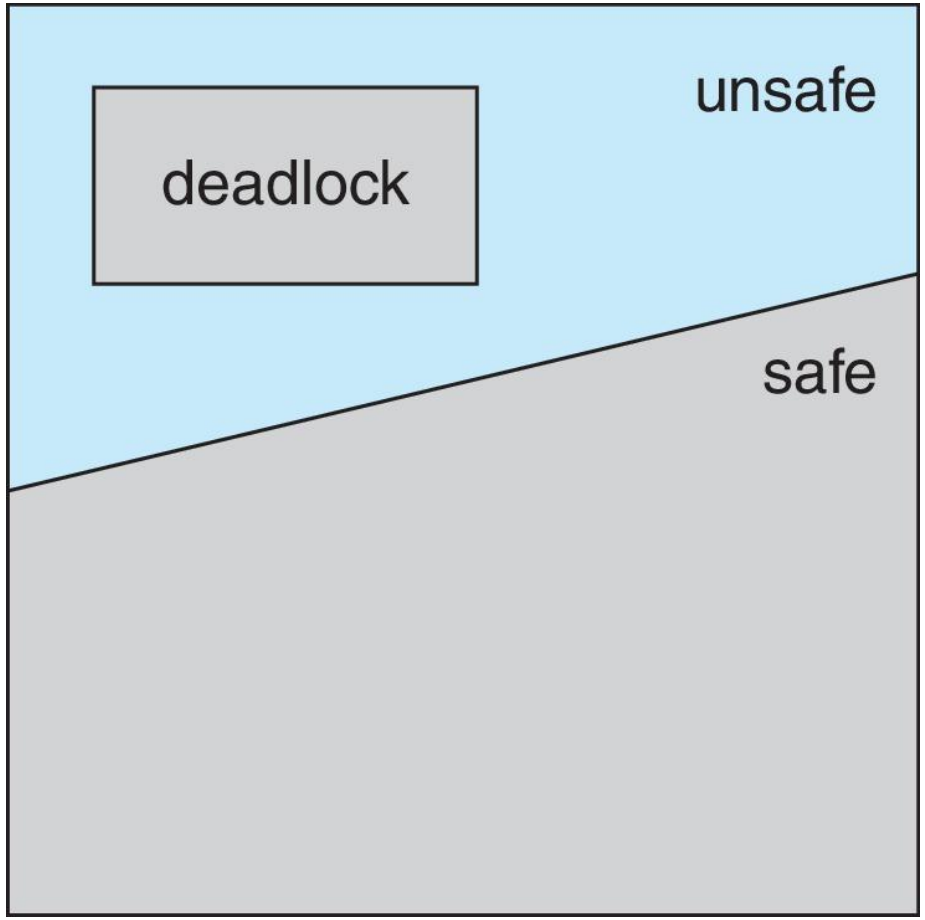
Basic Facts

If a system is in safe state $\Rightarrow$ no deadlocks

If a system is in unsafe state $\Rightarrow$ possibility of deadlock

Avoidance $\Rightarrow$ ensure that a system will never enter an unsafe state.

Safe, Unsafe, Deadlock State



Avoidance Algorithms

Single instance of a resource type

Use a resource-allocation graph

Multiple instances of a resource type

Use the Banker's Algorithm

Resource-Allocation Graph Scheme

Claim edge T_i --- R_j indicated that process T_j may request resource R_j; represented by a dashed line

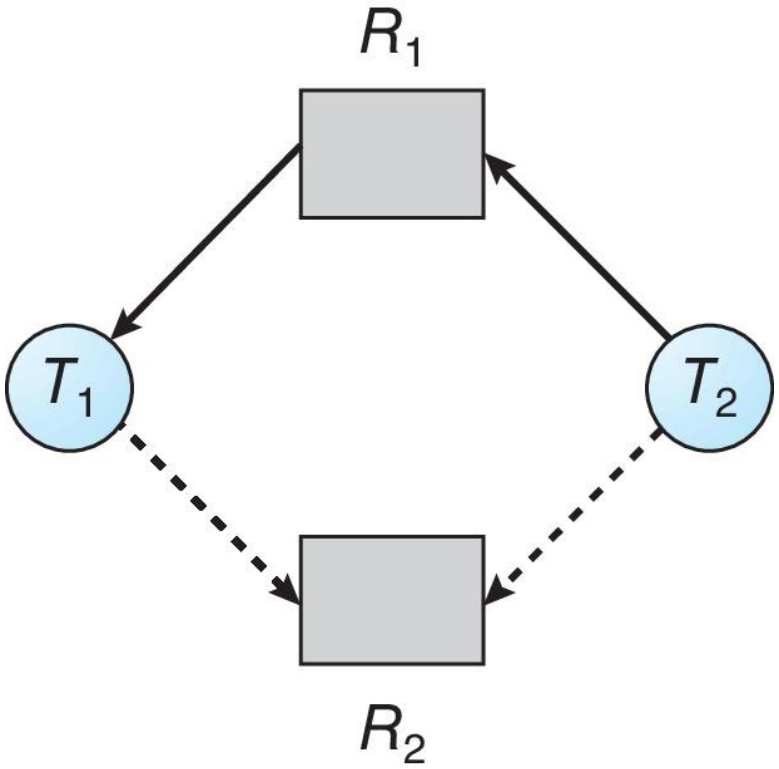
Claim edge converts to request edge when a thread requests a resource

Request edge converted to an assignment edge when the resource is allocated to the thread, from resource to thresd (process)

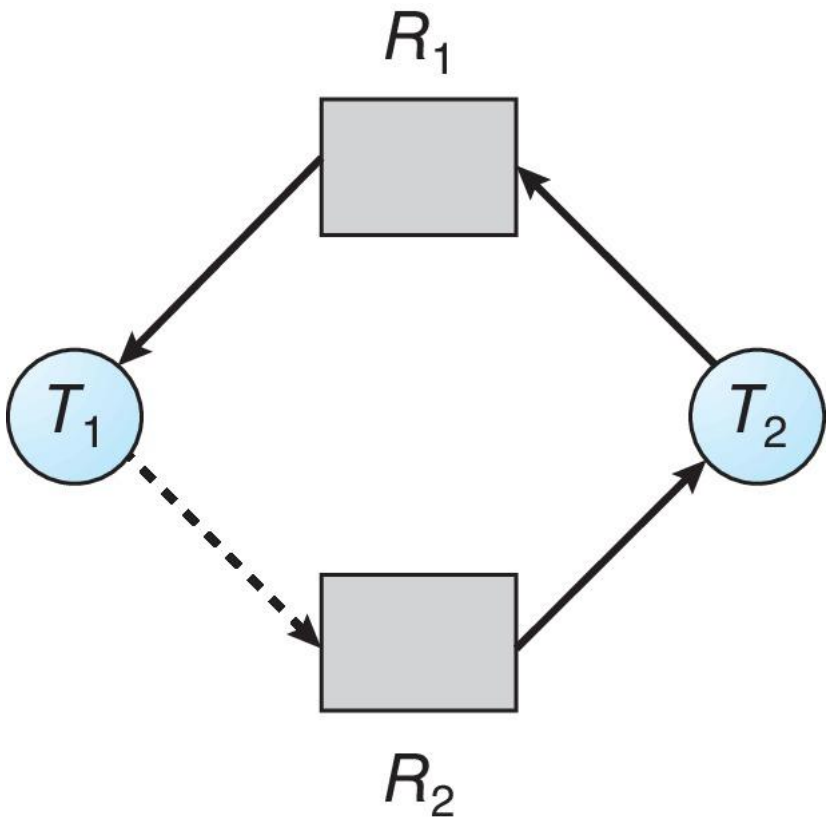
When a resource is released by a thread, assignment edge reconverts to a claim edge

Resources must be claimed *a priori* in the system

Resource-Allocation Graph



Unsafe State In Resource-Allocation Graph



Resource-Allocation Algorithm

Processes request shared resources like printers, scanners, memory, etc. The OS decides how to allocate them.

In a computer lab:
There are 2 printers.
Multiple students (processes) need to print assignments.
A student can only print if a printer is free.
If all printers are taken, others must wait.

Resource request
Hold and wait condition
What happens if no preemption is allowed
How bad allocation leads to deadlock

Resource-Allocation Algorithm

Suppose that thread T_i requests a resource R_j
The request can be granted only if converting the request edge to an assignment edge does not result in the formation of a cycle in the resource allocation graph

Banker’s Algorithm “Loan Approval in a Bank Analogy”

Think of processes as customers, resources as money, and the operating system as a bank that doesn’t want to go bankrupt.

Real time example:
You’re a banker. Customers (processes) apply for loans (resources).
You only approve a loan if, **after giving it**, the bank still has **enough money left** to handle **other potential demands**.

Banker Algorithm contd…

Each customer tells the bank the maximum amount of money they may need.
They may not ask for all of it at once.
The bank only gives out a loan if it ensures that:
The bank can still support all other customers to get their maximum money eventually.
No risk of running out of funds → safe state.

Banker's Algorithm "Loan Approval in a Bank Analogy"

Multiple instances of resources
Each thread must a priori claim maximum use
When a thread requests a resource, it may have to wait
When a thread gets all its resources it must return them in a finite amount of time